REPORT ON THOUGHT PROCESS AND WORK ACTIVITIES

STUDENT: Hena Šehović

1.Working Environment

I decided to work in Logisim, and before I start working, I have to do research about it. I watched a few videos so I can understand better the environment. This research I did before the actual project, so I did not only research about the components that I needed for the Traffic Light. Videos that I watched are:

https://youtu.be/cMz7wyY_PxE?si=RU-ANlFgjQJ-OmIp

https://youtu.be/jhtIKnH4a0c?si=JRMPFgZ1IyZcXe4D

https://youtu.be/n5erd_vs9l8?si=lvqM79ab_Y5i91Pp

https://youtu.be/UEKSLb20ARw?si=TQ0xjV96bqd1TRKj

https://faculty.kfupm.edu.sa/COE/mayez/ps-coe308/Project/The%20Guide%20to%20Being%20a%20Logisim%20User.pdf

Additional source of information was my friend who worked previously in Logisim, and she helped me when I encountered certain problems.

2. Project Logic

First, I am going to think through the project. Design the project logic and see what components I need. This part I did on the paper. But here I wrote down everything that I did before I made this project.

What are we trying to do?

We want to model a machine that represents the behaviour of a traffic light, our first step is to analyse the states that the traffic light goes through. The Traffic light is made from a cycle that repeats, and that cycle is made from four states, where each state represents a specific combination of lights being on. The four mentioned states are:

A. Red light is on
B. Both yellow and red light are on
C. Green light is on
D. Yellow light is on

Now we are representing each state:

State	Code	Red	Yellow	Green
A	00	1	0	0
B	01	1	1	0
C	11	0	0	1
D	10	0	1	1

Now we need to make a table for the way states transition. We are going to use T Flip-Flops, because of their simplicity and efficiency in implementing state transitions. They toggle outputs based on the current input, which aligns well with the cyclic nature of traffic light operations, where states need to transition sequentially and predictably.

Transition table:

Q1	Q0	Q1_next	Q0_next	T1	T0
0	0	0	1	0	1
0	1	1	1	1	0
1	1	1	0	0	1
1	0	0	0	1	0

K' Map for T1:

T1	Q1	0	1
Q0	0		1
	1	1	

T1=Q1'Q0+Q1QO'=Q1⊕Q0

K' Map for T2:

T0	Q1	0	1
Q0	0	1	
	1		1

T1=Q1'Q0'+Q1Q0=(Q1⊕Q0)'

The outputs of the system (e.g., which lights are active) are directly tied to the state that they are currently in. For example, in state A (Q1 = 0, Q0 = 0), only the red light is on, in state B

(Q1 = 0, Q0 = 1), both red and yellow lights are on. This relationship is previously defined and does not rely on external inputs to decide the outputs. So, that means that outputs depend only on the current state. This simplifies the design and implementation. Since each state maps directly to specific outputs, we can predict the system's behaviour easily by knowing its current state.

Q1	Q0	Red	Yellow	Green
0	0	1	0	0
0	1	1	1	0
1	1	0	0	1
1	0	0	1	0

Red=Q1' (*we do not care what Q0 is)

Yellow=Q1'Q0+Q1Q0'= Q1⊕Q0

Green=Q1Q0

Now we are going to design a traffic light system where each state has a specific duration. The traffic light will cycle through four primary states with the following timing:

A. Red light is on for 20 seconds
B. Both yellow and red light are on for 4 seconds
C. Green light is on for 20 seconds
D. Yellow light is on for 4 seconds

To simplify our timing and calculations, we set the clock period T=4 seconds. This clock period determines how we measure the duration of each state in terms of clock periods, as follows:

A. Red light is on for 5 periods
B. Both yellow and red light are on for 1 period
C. Green light is on for 5 periods
D. Yellow light is on for 1 period

When we sum up the periods for all states, the total cycle duration of the traffic light system is 5+1+5+1=12 periods. Each period corresponds to a distinct state in the finite-state machine controlling the traffic light.

To represent the states, we use a 4-bit binary code. The table below provides the details of each state, its binary code, the active light(s), and the corresponding light statuses for red, yellow, and green:

State	Code	Light	Red	Yellow	Green
S0	0000	Red	1	0	0
S1	0001	Red	1	0	0
S2	0010	Red	1	0	0
S3	0011	Red	1	0	0
S4	0100	Red	1	0	0
S5	0111	Red + Yellow	1	1	0
S6	1000	Green	0	0	1
S7	1001	Green	0	0	1
S8	1010	Green	0	0	1
S9	1011	Green	0	0	1
S10	1100	Green	0	0	1
S11	1111	Yellow	0	1	0

We use T Flip Flop:

Q3	Q2	Q1	Q0	Q3_next	Q2_next	Q1_next	Q0_next	T3	T2	T1	T0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	1
0	0	1	1	0	1	0	0	0	1	1	1
0	1	0	0	0	1	1	1	0	0	1	1
0	1	0	1	X	X	X	X	X	X	X	X
0	1	1	0	X	X	X	X	X	X	X	X
0	1	1	1	1	0	0	0	1	1	1	1
1	0	0	0	1	0	0	1	0	0	0	1
1	0	0	1	1	0	1	0	0	0	1	1
1	0	1	0	1	0	1	1	0	0	0	1
1	0	1	1	1	1	0	0	0	1	1	1
1	1	0	0	1	1	1	1	0	0	1	1
1	1	0	1	X	X	X	X	X	X	X	X
1	1	1	0	X	X	X	X	X	X	X	X
1	1	1	1	0	0	0	0	1	1	1	1

Q3, Q2, Q1, Q0 - these represent the current state of the flip-flops (4-bit binary number), which encodes the current state of the traffic light system.

Q3_next, Q2_next, Q1_next, Q0_next - these represent the next state of the flip-flops, determined by the current state and the system's logic.

T3, T2, T1, T0 - these are the **T inputs** for each flip-flop. A T Flip-Flop toggles its output Q when T=1 and holds its current state when T=0. The values in these columns determine whether each flip-flop toggles or not during the transition to the next state.

Now, we will use Karnaugh Map to minimize:

T3	**Q1 Q0**	00	01	11	10
Q3Q2	00				
	01		X	1	X
	11		X	1	X
	10				

T3=Q2Q0

T2	**Q1 Q0**	00	01	11	10
Q3Q2	00			1	
	01		X	1	X
	11		X	1	X
	10			1	

T2=Q1Q0

T1	**Q1 Q0**	00	01	11	10
Q3Q2	00		1	1	
	01	1	X	1	X
	11	1	X	1	X
	10		1	1	

T1=Q2+Q0

T0	**Q1 Q0**	00	01	11	10
Q3Q2	00	1	1	1	1
	01	1	X	1	X
	11	1	X	1	X
	10	1	1	1	1

T0=1

These minimized expressions can now be implemented in the circuit to control the flip-flops and generate the required state transitions efficiently.

The state of the traffic light automation depends entirely on its current state, represented by the binary values Q3, Q2,Q1,Q0. These bits encode the light signals (Red, Yellow, Green), determining their on/off state. This relationship allows us to simplify the logic for each light using Karnaugh Maps (K-Maps). Below, I have broken down the process and explained step by step:

Q3	Q2	Q1	Q0	Red	Yellow	Green
0	0	0	0	1	0	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	1	X	X	X
0	1	1	0	X	X	X
0	1	1	1	1	1	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	0	1
1	0	1	1	0	0	1
1	1	0	0	0	0	1
1	1	0	1	X	X	X
1	1	1	0	X	X	X
1	1	1	1	0	1	0

We use Karnaugh Maps to minimize the logic for turning each light on or off. We simplify the logic for each light, reducing the number of gates required in the circuit.

The K-Map for the Red light is:

Red	Q1 Q0	00	01	11	10
Q3Q2	00	1	1	1	1
	01	1	X	1	X
	11		X		X
	10				

Minimized expression: Red=Q3’

The K-Map for the Yellow light is:

Yellow	Q1 Q0	00	01	11	10
Q3Q2	00				
	01		X	1	X

	11		X	1	X
	10				

Minimized expression: Yellow=Q2Q0

The K-Map for the Green light is:

Green	**Q1 Q0**	00	01	11	10
Q3Q2	00				
	01		X		X
	11	1	X		X
	10	1	1	1	1

Minimized expression: Green=Q3Q2'+Q3Q1'=Q3(Q2'+Q1') =Q3(Q2Q1)'

After doing all of this I realized that I wanted to add two more things. I wanted to add an enable, so we can turn the traffic light working on or off. Also, I wanted to add another option for enabling the alternative mode, when traffic light is not working for certain reason, and in that scenario, the traffic lights will blink on and off. So, after all of this we have to add some new things and change previous ones. Everything that I did previously will be prototype 1, and after adding these two new components, for advancement, this I will call the prototype 2.

So, overview, I decided to add:

1. Enable functionality: A switch to turn the entire traffic light system on or off.

2. Alternative mode: A mode where all lights blink on and off (flashing mode) when the traffic light is not working properly.

Prototype 2 introduces more states and transitions to handle the added complexity of enabling and alternative modes. So, we must add that.

Coding the states:

Description	**State**	**Code**	**Red**	**Yellow**	**Green**
Starting state Enable=0	**A**	111	0	0	0

System working “normal”	**B**	000	1	0	0
	C	001	1	1	0
	D	011	0	0	1
	E	010	0	1	0
System working “alternatively”	**F**	100	1	1	1
	G	101	0	0	0

Current state	Transition Vector [ENABLE, ALT_MODE]	Next state
A	00	A
	01	A
	10	B
	11	F

Current state	Transition Vector [ENABLE, ALT_MODE]	Next state
B	00	A
	01	A
	10	C
	11	F

Current state	Transition Vector [ENABLE, ALT_MODE]	Next state
C	00	A
	01	A
	10	D
	11	F

Current state	Transition Vector [ENABLE, ALT_MODE]	Next state
D	00	A
	01	A
	10	E
	11	F

Current state	Transition Vector [ENABLE, ALT_MODE]	Next state
E	00	A
	01	A
	10	B

	11	F

Current state	Transition Vector [ENABLE, ALT_MODE]	Next state
F	00	A
	01	A
	10	B
	11	G

Current state	Transition Vector [ENABLE, ALT_MODE]	Next state
G	00	A
	01	A
	10	B
	11	F

EN	ALT	Q2	Q1	Q0	Q2_NEXT	Q1_NEXT	Q0_NEXT	D2	D1	D0
0	0	0	0	0	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1
0	0	0	1	0	1	1	1	1	1	1
0	0	0	1	1	1	1	1	1	1	1
0	0	1	0	0	1	1	1	1	1	1
0	0	1	0	1	1	1	1	1	1	1
0	0	1	1	0	X	X	X	X	X	X
0	0	1	1	1	1	1	1	1	1	1
0	1	0	0	0	1	1	1	1	1	1
0	1	0	0	1	1	1	1	1	1	1
0	1	0	1	0	1	1	1	1	1	1
0	1	0	1	1	1	1	1	1	1	1
0	1	1	0	0	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	1
0	1	1	1	0	X	X	X	X	X	X
0	1	1	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	1	0	0	1
1	0	0	0	1	0	1	1	0	1	1
1	0	0	1	0	0	0	0	0	0	0
1	0	0	1	1	0	1	0	0	1	0
1	0	1	0	0	0	0	0	0	0	0
1	0	1	0	1	0	0	0	0	0	0
1	0	1	1	0	X	X	X	X	X	X
1	0	1	1	1	0	0	0	0	0	0
1	1	0	0	0	1	0	0	1	0	0
1	1	0	0	1	1	0	0	1	0	0
1	1	0	1	0	1	0	0	1	0	0
1	1	0	1	1	1	0	0	1	0	0
1	1	1	0	0	1	0	1	1	0	1

1	1	1	0	1	1	0	0	1	0	0
1	1	1	1	0	X	X	X	X	X	X
1	1	1	1	1	1	0	0	1	0	0

D2=En'+Alt

D1=En'+Alt'Q2Q0

D0=En'+ Alt'Q2'Q1'+AltQ2Q1'Q0'

Q2	Q1	Q0	Red	Yellow	Green
0	0	0	1	0	0
0	0	1	1	1	0
0	1	0	0	1	0
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	1	0	X	x	x
1	1	1	0	0	0

Karnaugh Map for the red light:

R	**Q2**	0	0	1	1
	Q1	0	1	1	0
Q0	0	1		X	1
	1	1			

Red=Q2'Q1'+Q2Q0'

Karnaugh map for the yellow light:

Y	**Q2**	0	0	1	1
	Q1	0	1	1	0
Q0	0		1	X	1
	1	1			

Yellow=Q0'Q1+Q0'Q2+Q2'Q1'Q0

Karnaugh map for the green light:

G	Q2	0 0	0 1	1 1	1 0
	Q1				
Q0	0			X	1
	1		1		

Green=Q0’Q2+Q2’Q1Q0

Now that we have discussed the logic. I must implement all of this into our simulation. To test, if everything is working properly and to see our simulation.

3. Writing everything in Logisim

Which components do I need?

Clock Signal: Used to generate a clock pulse that drives the sequential behavior of the circuit.

Enable Input: A switch or input pin labelled "enable" controls whether the circuit is active or not.

D Flip-Flops: Three D flip-flops are used for storing the state of the system. These flip-flops are fundamental for sequential logic and state transitions. (Prototype 2)

T Flip-Flops: Four T flip-flops are used for storing the state of the system (Prototype 1 and Prototype 3)

Logic Gates:

AND Gates: Several AND gates are used to ensure that specific conditions must be met simultaneously for a signal to propagate.

OR Gates: OR gates combine multiple input signals, allowing the activation of lights based on various conditions.

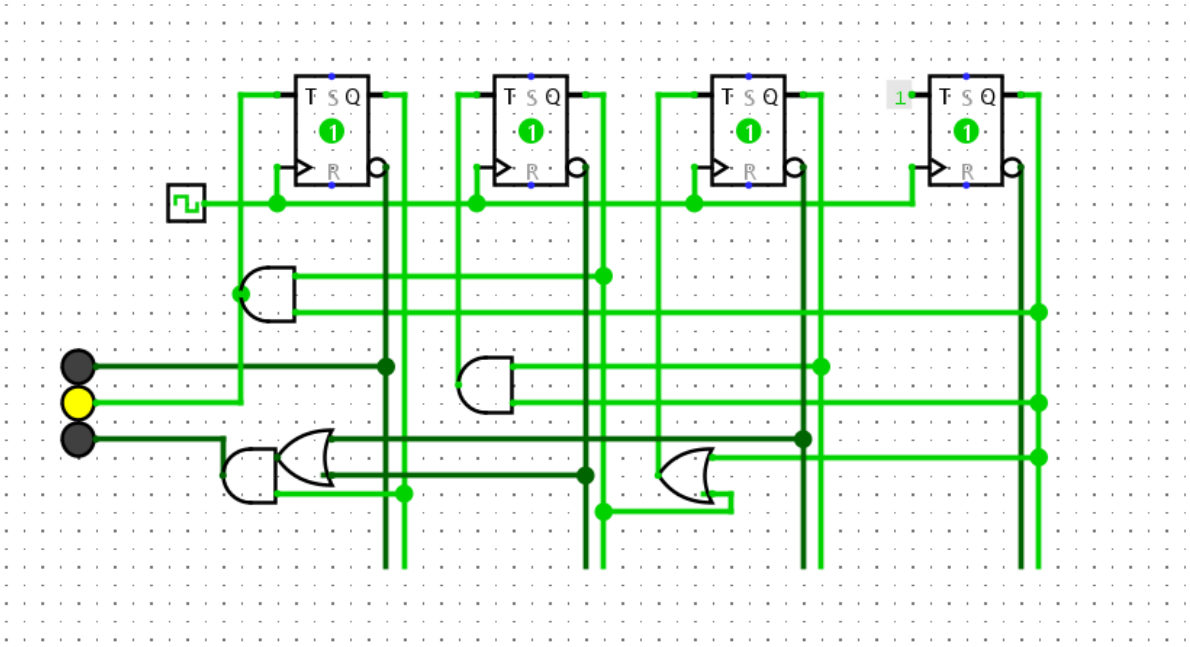
NOT Gates (Inverters): NOT gates invert signals, ensuring proper logical conditions for transitions.

LEDs: Three LEDs (Red, Yellow, Green) represent the traffic light outputs. The LEDs visually indicate the current state of the system.

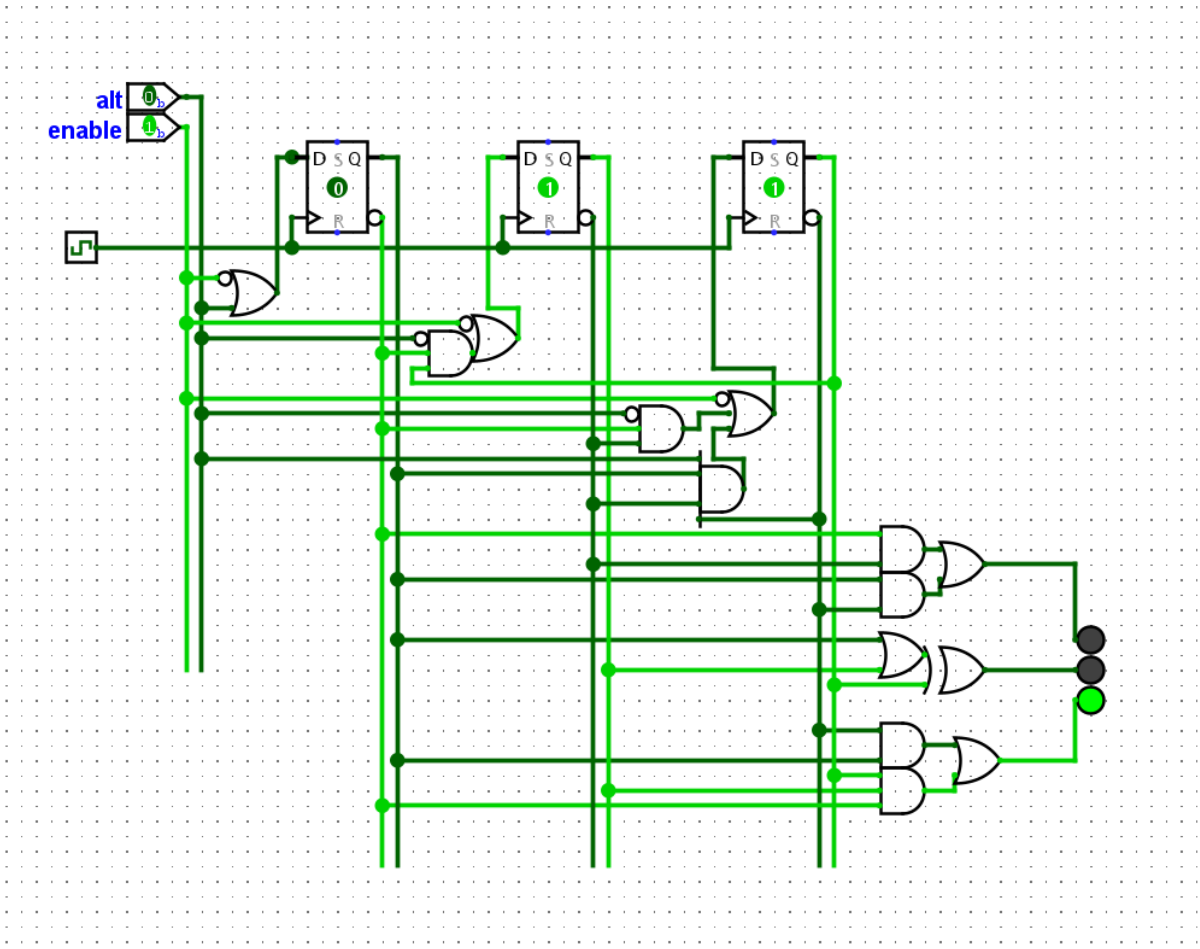
Connecting Wires: Wires establish logical connections between the components and ensure proper flow of signals.

Input Pins (Alt/Enable): These control inputs (like "alt") influence the state of the system or provide alternate signals.

I first designed the Prototype 1:

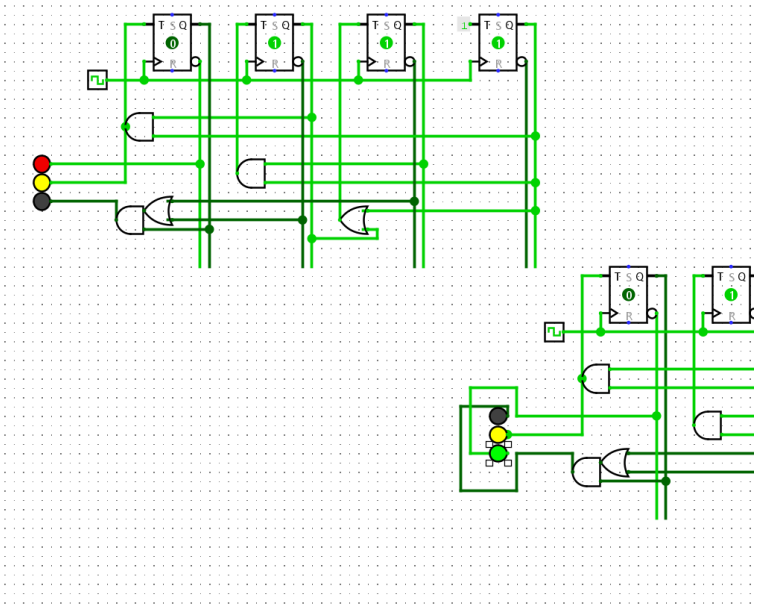


And then the Prototype 2:



4. Fixing the Project

After all this work I realised that I did not read the file that professor sent properly. I missed the crucial part that it should be a two-way intersection. This is the biggest problem that I encounter, because at this point, I was already tired of the project. I did the report, I finished the stream of thought report. I started panicking because I thought that I had to change the whole project, everything. But then I started thinking and realised that maybe I could add another traffic light, and just change the wires, the wire that was going to the red LED in first traffic light, I put in the green LED of the second traffic light, interchange the other from green to red, and for the yellow LED in the first traffic light I put in the yellow LED of the second traffic light. I started the simulation and quickly realized that in the second traffic light the green light was switching on together with the yellow one.



Next question was. How to fix that?

I first tried on the Prototype 1 to add the second traffic light. I had to go back to the table of my states in Prototype 1 and change them for the second traffic light. The states for the second light should look like this:

Q3	Q2	Q1	Q0	Red	Yellow	Green
0	0	0	0	0	0	1
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	0	0	1
0	1	0	1	X	X	X
0	1	1	0	X	X	X
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	1	0	0

1	0	1	1	1	0	0
1	1	0	0	1	0	0
1	1	0	1	X	X	X
1	1	1	0	X	X	X
1	1	1	1	1	1	0

The K-Map for the Red light is:

Red	**Q1 Q0**	00	01	11	10
Q3Q2	00				
	01		X		X
	11	1	X	1	X
	10	1	1	1	1

Minimized expression: Red=Q3

The K-Map for the Yellow light is:

Yellow	**Q1 Q0**	00	01	11	10
Q3Q2	00				
	01		X	1	X
	11		X	1	X
	10				

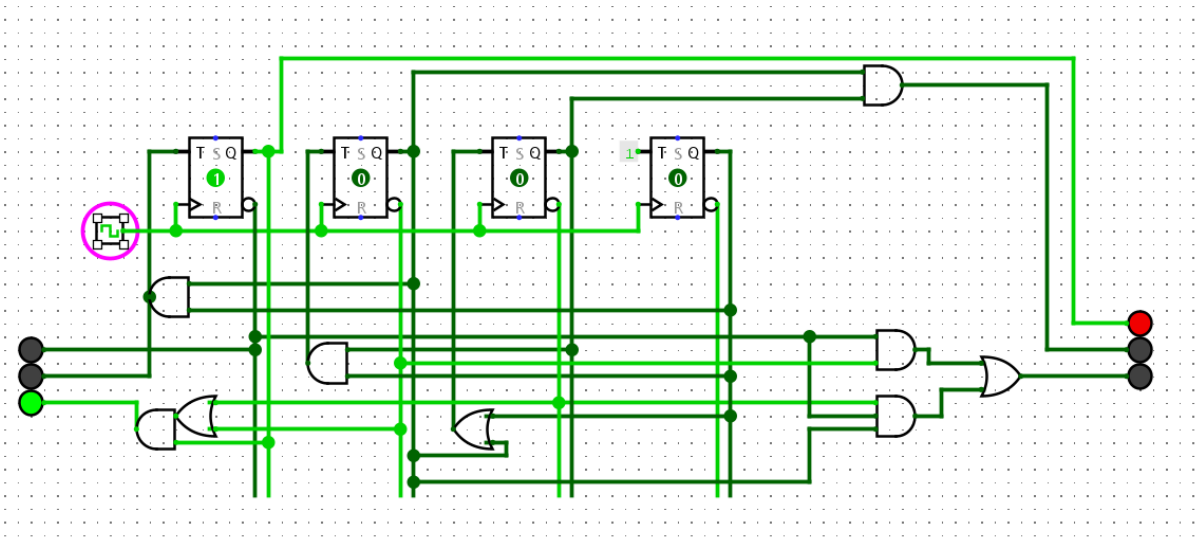
Minimized expression: Yellow=Q2Q0

The K-Map for the Green light is:

Green	**Q1 Q0**	00	01	11	10
Q3Q2	00	1	1	1	1
	01	1	X		X
	11		X		X
	10				

Minimized expression: Green=Q3’Q2’+Q1’Q3’Q2

So now that I have these expressions, I use them to connect the second traffic light that looks like this:



This is the Prototype 3.

After this I again decided that I am going to add the second traffic light to the system with alternative way of working and enable switch. I again applied changes to the state table for the second traffic light, and I got this:

Q2	Q1	Q0	Red	Yellow	Green
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	1	1	0
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	0	0	0
1	1	0	X	x	x
1	1	1	0	0	0

We use Karnaugh map to minimize the expressions:

Karnaugh Map for the red light:

R	Q2 / Q1	0 0	0 1	1 1	1 0
Q0	0		1	X	1
	1		1		

Red=Q2’Q1+Q0’Q2

Karnaugh map for the yellow light:

Y	**Q2**	0	0	1	1
	Q1	0	1	1	0
Q0	0		1	X	1
	1	1			

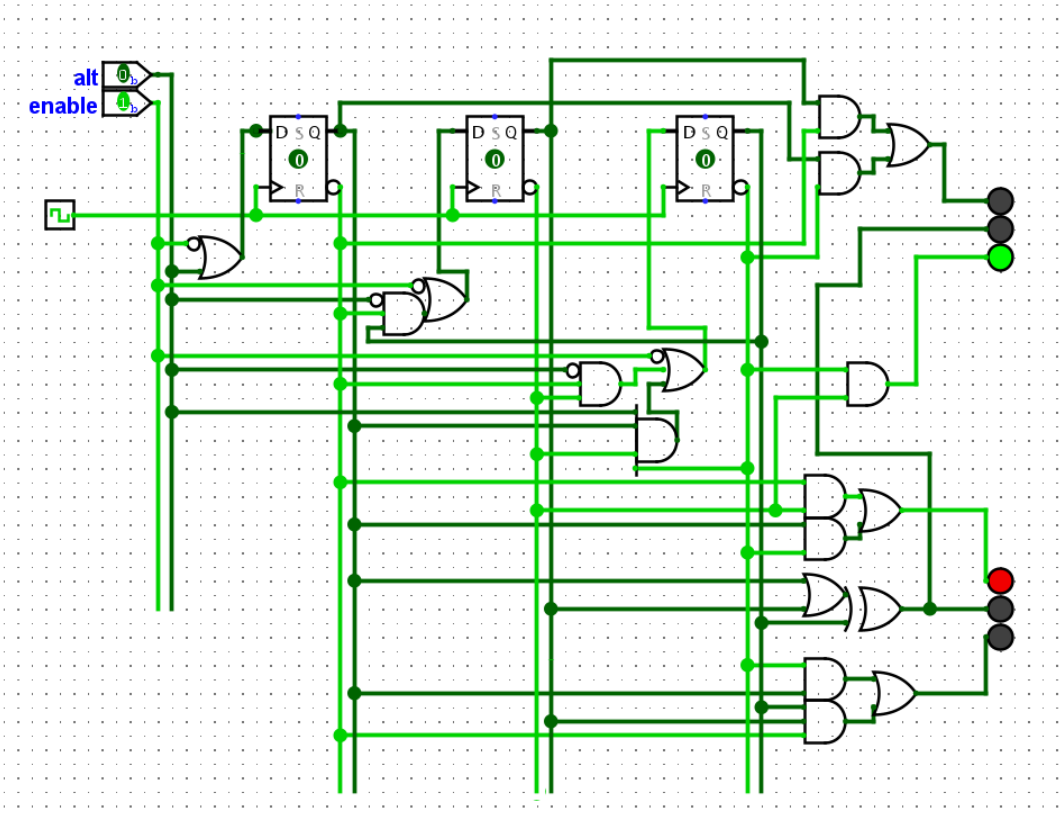
Yellow=Q0'Q1+Q0'Q2+Q2'Q1'Q0

Karnaugh map for the green light:

G	**Q2**	0	0	1	1
	Q1	0	1	1	0
Q0	0	1		X	1
	1				

Green=Q0'Q1'

After I applied these logic gates in Logisim for my second traffic light, I got this:



This is the Prototype 4 and the final product